

Interface Template

Page Content

- [Concept](#)
- [Building Blocks of an Interface](#)
- [Usage of an interface](#)
- [Example: NIC interface if_OS_NIC](#)

Concept

- an interface should always be agnostic of any component that implements it
- no component should "own" an interface, components should always just use existing interfaces interface, but neither bend nor abuse them for specific needs.
- creating another component that also implements an interface should be possible by re-using the interface definitions. The new component does not need anything from the old component concerning the interface.
- a component can implement an interface multiple times, it can even implement both sides of the interface at the same time - and this multiple times.
- an interface is actually a grouping of one or more primitive CAMkES interfaces (RPC, dataport, event), where the grouping has a unique name within one component
- ⚠ all this is still a hacky work-around, as we should actually consider creating our own CAMkES connectors for the interfaces we need.

Building Blocks of an Interface

- Define the RPC functions
- define macros for both sides of the interface, e.g.
 - client side and server side
 - upper side and lower side
 - left and right side
- define macros to connect two components via the interface
 - this can be multiple macros, depending on how the interface actually looks like and is used practically

Usage of an interface

- each interface of a component has a type and a name.
- multiple usages of an interface must use different names (or prefixes) then.
- Example:
 - the interface IF_FOO is a grouping of the primitive CAMkES interfaces a "rpc" and "dataport"
 - a component implementing IF_FOO will have the fields "<name>_rpc" and "<name>_port", where <name> is an arbitrary name within the component the IF_FOO interface
 - if the component implements IF_FOO twice, it will have the fields
 - <name1>_rpc, <name1>_port
 - <name2>_rpc, <name2>_port
 - if the component implements IF_FOO twice and also an interface IF_BAR that is a grouping of the primitive CAMkES interfaces a "event" and "dataport", it will have the fields
 - IF_FOO #1: <name1>_rpc, <name1>_port
 - IF_FOO #2: <name2>_rpc, <name2>_port
 - IF_BAR: <name3>_event, <name3>_port

Example: NIC interface if_OS_NIC

The interface header provides all necessary macros. The interface consists of the 2 primitive CAMkES interface "rpc", "dataport" and "event". The event has a special handling because there are two usage scenarios.

- components have one dedicated event handler for every interface they provide
- components have only a shared event handler for the events of multiple interfaces.

if_OS_NIC.h

```
//-----  
// The RPC interface of if_OS_Nic.  
procedure if_OS_Nic {  
    ...  
}
```

```

//=====
// Component interface fields macros
//=====

//-----
// The interface fields of a component implementing the upper side of the NIC
// interface, except for the event. The event is excluded because the upper side
// component can have multiple event sources connect to one event sink.
#define IF_OS_NIC_UPPER_INTERFACE_FIELDS_WITHOUT_EVENT(_prefix_, ...)
\
    uses      if_OS_Nic      _prefix_##_rpc; \
    dataport  Buf            _prefix_##_port_to; \
    ...

//-----
// The interface fields of a component implementing the upper side of the NIC
// interface.
#define IF_OS_NIC_UPPER_INTERFACE_FIELDS(_prefix_, ... ) \
\
    consumes  EventDataAvailable _prefix_##_event_hasData; \
    FIELDS_IF_OS_NIC_UPPER_WITHOUT_EVENT(_prefix_, ...)

//-----
// Fields of a component implementing the lower side of the NIC interface.
#define IF_OS_NIC_LOWER_INTERFACE_FIELDS(_prefix_, ...) \
\
    provides  if_OS_Nic      _prefix_##_rpc; \
    ...

//=====
// component interface field connection macros
//=====

//-----
// Connect two components via the NIC interface, but leave out the connection
// for event. Rationale is, that often multiple event sources are connected to
// the same event sink. This can be done with IF_OS_NIC_CONNECT_EVENT() then.
#define IF_OS_NIC_CONNECT_WITHOUT_EVENT( \
    _inst_upper_, \
    _inst_upper_field_prefix_, \
    _inst_lower_, \
    _inst_lower_field_prefix_) \
\
    connection sel4RPCCall \
        conn_##_inst_upper_##_inst_lower_##_rpc( \
            from _inst_upper._inst_upper_field_prefix_##_rpc, \
            to   _inst_lower._inst_lower_field_prefix_##_rpc); \
\
    ...

//-----
// Connect the NIC event between two components.
#define IF_OS_NIC_CONNECT_EVENT( \
    _inst_upper_, \
    _inst_upper_event_, \
    _inst_lower_, \
    _inst_lower_field_prefix_) \
\
    connection sel4NotificationNative \
        conn_##_inst_lower_##_inst_upper_##_event_hasData( \
            from _inst_lower._inst_lower_field_prefix_##_event_hasData, \
            to   _inst_upper._inst_upper_event_);

//-----

```

```

// Connect two components via the NIC interface, including the event.
#define IF_OS_NIC_CONNECT( \
    _inst_upper_, \
    _inst_upper_field_prefix_, \
    _inst_lower_, \
    _inst_lower_field_prefix_) \
    \
    CONNECT_IF_OS_NIC_WITHOUT_EVENT( \
        _inst_upper_. _inst_upper_field_prefix_, \
        _inst_lower_. _inst_lower_field_prefix_ \
    ) \
    \
    CONNECT_IF_OS_NIC_EVENT( \
        _inst_upper_, \
        _inst_upper_field_prefix_##_event_hasData, \
        _inst_lower_, \
        _inst_lower_field_prefix_)

```

A network stack implementation can use the NIC interface macro to implement the interface. IN the example below it chooses the "nic" as name (or <prefix>) of the interface .

```

#include <if_OS_Nic.camkes>

...

//-----
#define NwStack_COMPONENT_DEFINE(_name_, ...) \
    \
    component _name_ \
    { \
        control; \
        \
        consumes EventDataAvailable    event_tick_or_data; \
        \
        IF_OS_NIC_UPPER_INTERFACE_FIELDS_WITHOUT_EVENT(nic, ...) \
        \
        ...
    }

//-----
// Connect a network stack instance to a ticker event and a NIC to it. So far,
// all if_OS_Nic related fields of the network stack use the prefix 'nic', and
// the same prefix is also used by all network drivers for their if_OS_Nic
// related fields. As long a no NIC driver breaks this scheme (e.g. because it
// has multiple if_OS_Nic), there is no need to have an additional macro
// parameter here that passes the driver's prefix.
// additional macro parameter.
#define NwStack_INSTANCE_CONNECT( \
    _name_, \
    _ticker_event_src_, \
    _nic_) \
    \
    ...
    IF_OS_NIC_CONNECT_WITHOUT_EVENT( \
        _name_, \
        nic, \
        _nic_, \
        nic) \
    \
    IF_OS_NIC_CONNECT_EVENT( \
        _name_, \
        event_tick_or_data, \
        _nic_, \
        nic)

```

A second NIC interface could easily be added by using the macro IF_OS_NIC_UPPER_INTERFACE_FIELDS_WITHOUT_EVENT() a second time with e. g. "nic2" as name